

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Отчёт по домашней работе №5

«Разработка одностраничного веб-приложения (SPA) с использованием
фреймворка Vue.JS»

по дисциплине «Web-разработка: Frontend»
Семестр 4

Выполнил: студент
Таряник Антон
гр. J3211

Проверил: Добряков Д. И.

Санкт-Петербург
2026

Содержание

1	Задача	2
2	Ход работы	2
2.1	Инициализация проекта и зависимости	2
2.2	Подключение зависимостей в main.js	2
2.3	Настройка состояния через Pinia (Stores)	2
2.4	Компонентная архитектура и Composables	3
2.5	Роутинг и Navigation Guards	3
2.6	Работа с директивами Vue (v-model, v-for, v-if)	4
3	Результаты (Скриншоты)	4
4	Вывод	7

1 Задача

В рамках данной домашней работы необходимо изучить основные команды пакетного менеджера NPM и научиться стартовать проект на Vue.js. Требуется перенести ранее реализованный сайт платформы разметки данных (LabelFlow) с чистого HTML/CSS/JS на фреймворк Vue.js, следуя современным подходам разработки SPA: инициализация проекта, настройка зависимостей (axios, pinia, bootstrap), организация работы с состоянием, компонентная архитектура и роутинг.

2 Ход работы

2.1 Инициализация проекта и зависимости

Проект был инициализирован с помощью официального инструмента скаффолдинга Vite командой `npm create vue@latest`. В процессе инициализации были подключены Vue Router (для навигации) и Pinia (для управления состоянием).

Затем через NPM были установлены дополнительные зависимости, необходимые для работы логики и верстки приложения:

```
npm install axios bootstrap
```

2.2 Подключение зависимостей в main.js

Файл `src/main.js` является точкой входа. В нём происходит подключение глобальных стилей (Bootstrap и кастомные переменные), роутера, хранилища Pinia и базовой конфигурации Axios:

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'
import axios from 'axios'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
import './assets/variables.css'
import './assets/styles.css'

axios.defaults.baseURL = 'http://localhost:3000'

const app = createApp(App)

app.use(createPinia())
app.use(router)

app.mount('#app')
```

2.3 Настройка состояния через Pinia (Stores)

Для управления состоянием авторизации и пользовательскими данными был создан файл `src/stores/auth.js`. Хранилище содержит `state` (текущий пользователь, подтягиваемый из `localStorage`) и `actions` для взаимодействия с сервером через Axios:

```
import { defineStore } from 'pinia'
import axios from 'axios'

export const useAuthStore = defineStore('auth', {
  state: () => ({
    user: JSON.parse(localStorage.getItem('currentUser')) || null
  }),
  actions: {
    async login(email, password) {
      const { data } = await axios.get(`/users?email=${encodeURIComponent(email)}`)
      if (data.length > 0 && data[0].password === password) {
        this.user = data[0]
        localStorage.setItem('currentUser', JSON.stringify(this.user))
        return true
      }
      return false
    },
    logout() {
      this.user = null
      localStorage.removeItem('currentUser')
    }
  }
})
```

2.4 Компонентная архитектура и Composables

Проект был разделён на переиспользуемые компоненты (папка `components`) и страницы (папка `views`). Выделены следующие основные файлы:

- `AppNavbar.vue` — Навигационная панель для авторизованных пользователей.
- `GuestNavbar.vue` — Панель для лендинга и страниц входа/регистрации.
- `DashboardView.vue`, `SearchView.vue`, ... — Страницы приложения.

Повторяющаяся логика переключения тёмной/светлой темы вынесена в отдельный composable-файл `src/composables/useTheme.js`.

2.5 Роутинг и Navigation Guards

Настройка маршрутизатора выполнена в файле `src/router/index.js`. Для защиты страниц от неавторизованных пользователей используется `beforeEach` guard. Страницы подгружаются лениво (`lazy loading`) с помощью динамических импортов:

```
// Защита роутов
router.beforeEach((to, from, next) => {
  const authStore = useAuthStore()
  const isAuth = !!authStore.user

  if (to.meta.requiresAuth && !isAuth) {
    next({ name: 'login' })
  } else if ((to.name === 'login' || to.name === 'registration') && isAuth) {
    next({ name: 'dashboard' })
  } else {
    next()
  }
})
```

2.6 Работа с директивами Vue (v-model, v-for, v-if)

В представлениях активно применяются директивы фреймворка. Например, страница поиска (SearchView.vue) использует v-model для двустороннего связывания полей поиска и v-for для рендеринга карточек проектов:

```
<!-- Фрагмент SearchView.vue -->
<input v-model="searchQuery" type="text" class="form-control" placeholder="
  Введите название проекта...">

<div v-for="p in filteredProjects" :key="p.id" class="col-md-6 col-lg-4">
  <div class="card h-100 border-0 shadow-sm">
    <div class="card-body">
      <h5 class="fw-bold mb-2">{{ p.name }}</h5>
      <span :class="['badge mb-3', getBadgeStyle(p.type)]">{{ getTypeText(p.
type) }}</span>
      <router-link to="/annotation" class="btn btn-outline-primary btn-sm w
-100">Перейти</router-link>
    </div>
  </div>
</div>
```

3 Результаты (Скриншоты)

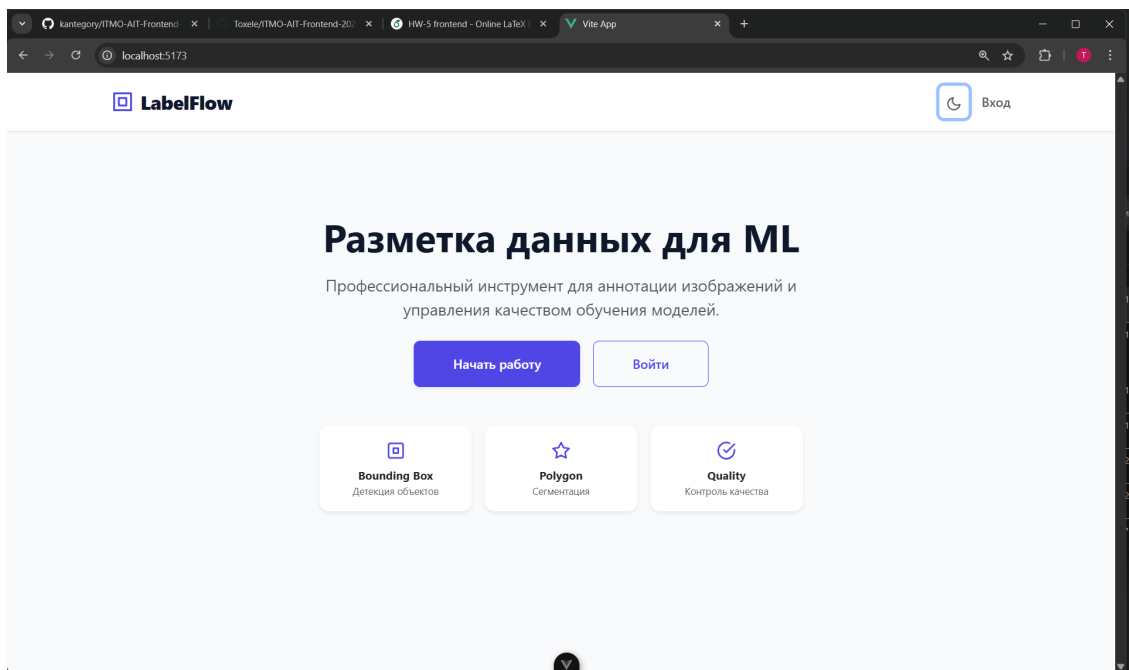


Рис. 1: Отображение страницы лендинга (Vue-компонент, белая тема)

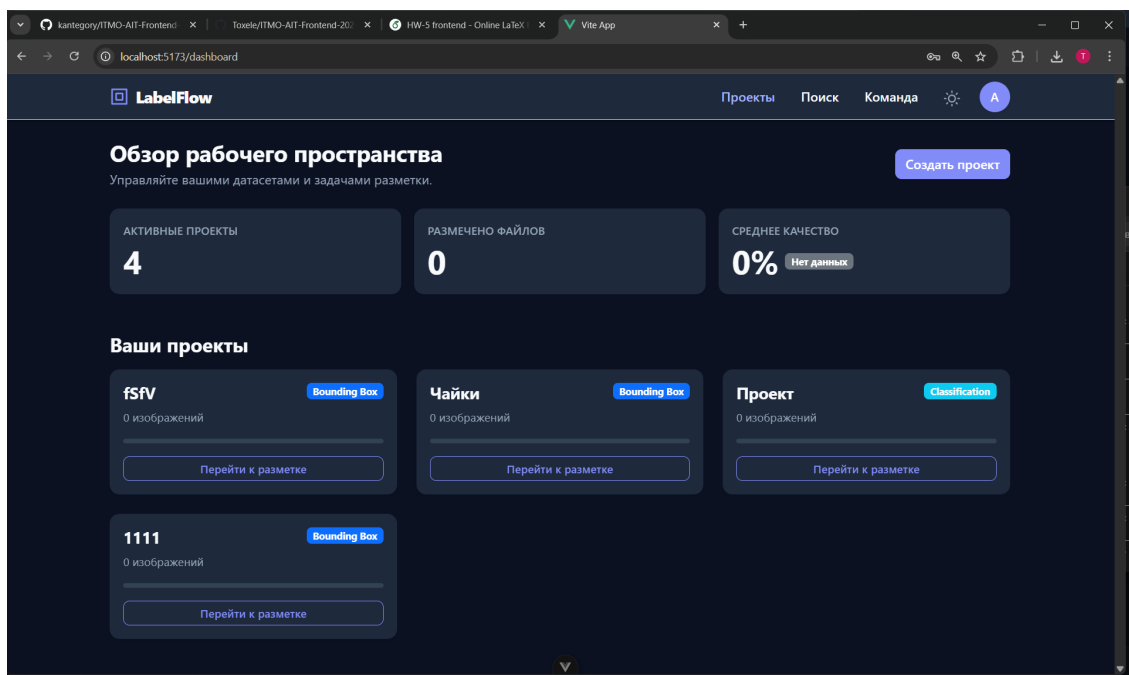


Рис. 2: Отображение страницы DashBoard (Vue-компонент, тёмная тема)

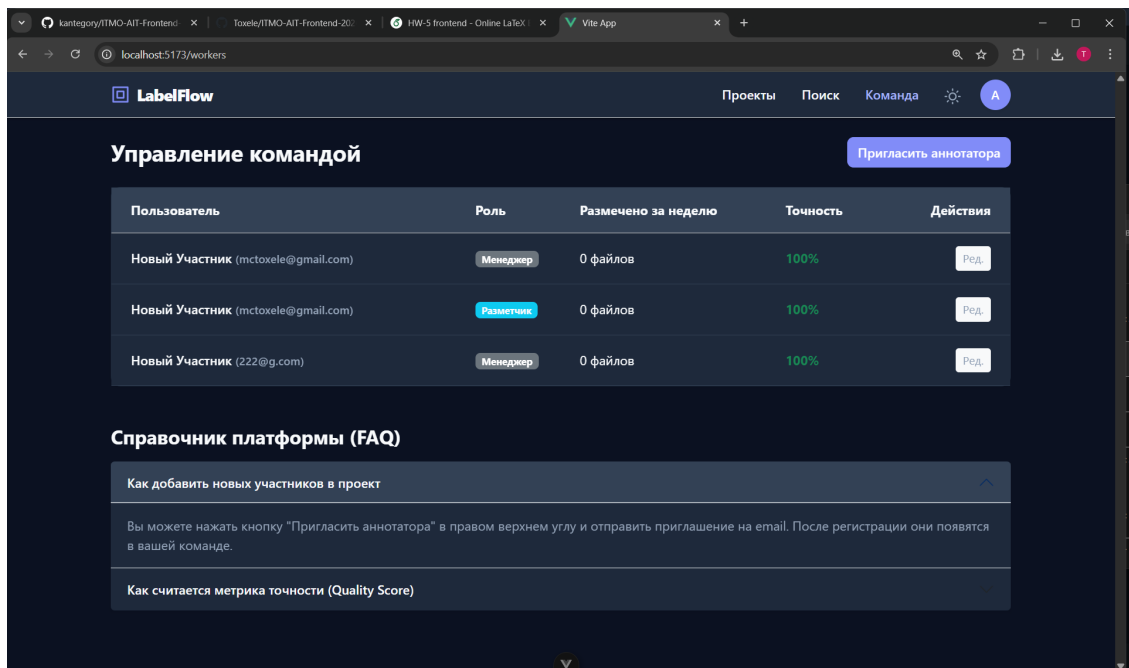


Рис. 3: Отображение страницы Workers (Vue-компонент, тёмная тема)

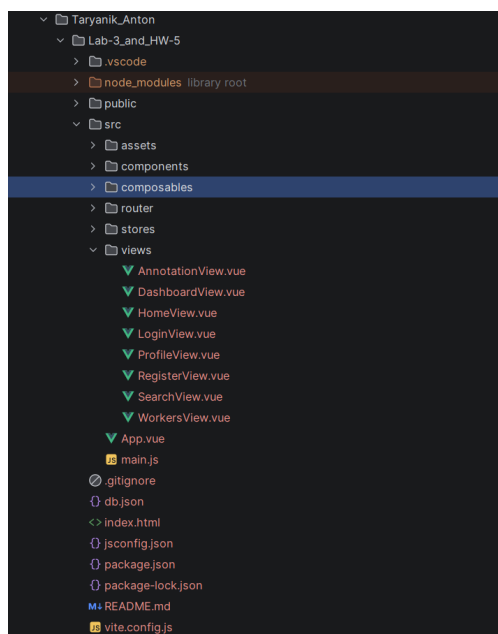


Рис. 4: Структура файлов Vue-проекта

4 Вывод

В ходе домашней работы интерфейс платформы полностью перенесён с Vanilla JS на фреймворк Vue.js. Были успешно освоены инструменты инициализации проекта (Vite, NPM), настройка зависимостей и архитектуры приложения.

Реализовано реактивное управление состоянием через Pinia, настроена защита маршрутов с помощью Vue Router, а прямые манипуляции с DOM (`innerHTML`) заменены на декларативный подход с использованием директив `v-model`, `v-for` и `v-if`. SPA-приложение запускается без ошибок и корректно взаимодействует с mock-сервером через Axios.